File Handling and Collections in Java

Object-Oriented Programming with Java — Week 7

Index

1. Learning Objectives — what you should be able to do before the exam stops being scary
2. Streams: Data on the Move — the pipes between your program and the outside world
3. File Handling Operations — the five things you can do to a file (besides losing it)
4. Byte Streams and Character Streams — raw bytes versus readable text
5. Writing Text Files with FileWriter — teaching your program to remember
6. Reading Text Files with FileReader — getting the memories back, one character at a time
7. Buffered Streams — carry the whole bag, not one apple at a time
8. Reading and Writing Binary Files — for data that is not text
9. Serialization — flat-pack an object now, rebuild it later
10. The transient Keyword — secrets that should stay out of the file
11. File Handling Summary and Exercises — cheat sheet plus homework
12. Why We Need Collections — arrays hit a wall at index 100
13. Java Collections Framework — the family tree of data structures
14. The Four Main Interfaces — List, Set, Queue, Map, and who gets to have duplicates
15. Generics — teaching a list to say no to footballs
16. Collections and Generics Summary — which container for which job
17. Practice Questions — five problems, zero solutions, full sympathy

---

1. Learning Objectives

By the end of these notes you should be able to:

File handling and input/output

- Explain how Java handles files using **streams**. In C, files are handled through library functions; in Java, they are handled through predefined (system-defined) classes grouped in packages.
- Read and write **text files** and **binary files**.
- Store and retrieve **objects** using serialization and deserialization.

Collections and generics

- Understand the Java Collections Framework and its architecture.
- Select appropriate collection classes based on application requirements.
- Use the `List`, `Set`, `Queue`, and `Map` interfaces.
- Implement `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, and `HashMap`.
- Traverse collections using iterators.
- Understand the importance of generics in type safety and code reuse.
- Work with wrapper classes for primitive data handling.

> **Note for Review:** The last three collection objectives (iterators, wrapper classes, and hands-on implementation of each class) build on the architecture and interfaces covered here. These notes lay the foundation: what each interface promises, which classes implement it, and why generics exist.

Part A — File Input/Output and File Handling

2. Streams: Data on the Move

The problem

A program's variables live in memory and vanish when the program ends. Suppose a program calculates the marks of every student. Unless it saves them somewhere, all results are lost at exit. The program also needs ways to *receive* data (keyboard, file, network) and to *send* data (screen, file, network).

The idea and the analogy

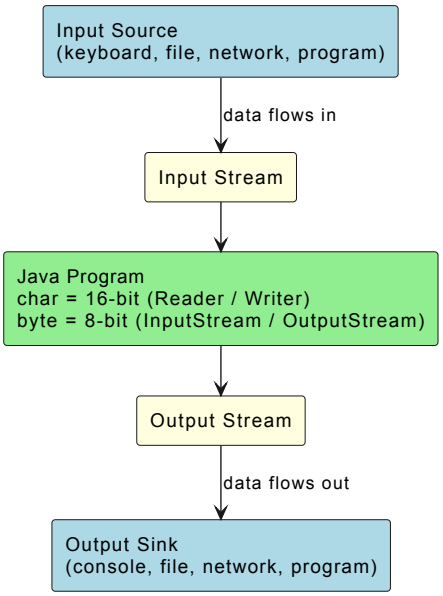
Think of a **water pipe**. Water (data) flows in one direction through a pipe between a tank (your program) and the outside world. One pipe brings water in; a different pipe carries it out.

> The analogy breaks down in one place: real pipes carry only one kind of liquid, but Java has two kinds of pipes (byte and character), chosen by what the data is.

The formal concept

A **stream** represents a flow of data between a program and an external source.

- **Input stream:** carries data **into** the program from an **input source** (keyboard, file, network, another program).
- **Output stream:** carries data **out of** the program to an **output sink** (console, file, network, another program).



Internal and external data formats

Data looks different inside the program than outside it.

	Formats
Internal (inside the Java program)	Text as `char` (the lecture calls this UCS-2), and primitive values such as `int`, `float`, `double`
External (in files, networks, etc.)	Text in various **encodings** (US-ASCII, ISO-8859-1, UCS-2, UTF-8, UTF-16, UTF-16BE, UTF-16LE), or **binary** (raw bytes)

An **encoding** is the agreed rule for converting characters into bytes (and back). Streams perform this conversion for you when the data is text.

> **Note for Review:** The lecture labels Java's internal `char` as UCS-2. Modern Java actually uses UTF-16, which matches UCS-2 for ordinary characters. For these notes the practical point is the same: a `char` is 16 bits wide.

The two sizes that matter:

$$\text{byte: } 2^8 = 256 \text{ values} \qquad \text{char: } 2^{16} = 65536 \text{ values}$$

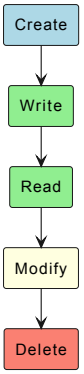
Two families of streams

	Character streams	Byte streams
Unit of data	`char` (16-bit)	`byte` (8-bit)
Base classes	`Reader` / `Writer`	`InputStream` / `OutputStream`
Used for	Text data	Binary data

3. File Handling Operations

File handling lets a program work with files that outlive the program. A program can:

Operation	Meaning
Create	Make a new file
Read	Fetch data from a file (read-only; nothing changes)
Write	Put data into a file
Modify	Change what is already stored
Delete	Remove the file



Java provides all of this through the **`java.io` package** ("io" = input/output).

The order above is the *typical* life of a file. The operations depend on each other (you read only after something exists), but a program may use them in any sensible order.

Importing the package

```
import java.io.*;
```

`import` makes classes from another package usable by their short names. `java.io` is the package, and `*` means "all classes in it". Without this line, writing `FileWriter` would be an error because the compiler would not know where that class lives.

Three packages appear repeatedly in Java:

Package	Purpose
`java.io`	Input and output (files, streams)
`java.lang`	Core language classes (imported automatically)
`java.util`	Utilities, including the Collections Framework (Part B)

Supplementary: creating and deleting with the `File` class

The lecture lists create and delete as operations. The class that represents a file *on disk* is `File`:

```
import java.io.*;

class FileOperations {
    public static void main(String args[]) throws IOException {
        File file = new File("data.txt");
        if (file.createNewFile()) {
            System.out.println("File created");
        } else {
            System.out.println("File already exists");
        }
        file.delete();
    }
}
```

- `new File("data.txt")` only creates a **Java object that stands for the path**. It does not create anything on disk.
- `createNewFile()` creates the actual file. It returns `true` if the file was created and `false` if it already existed. It can throw `IOException`, so `main` declares `throws IOException`.
- `delete()` removes the file.

4. Byte Streams and Character Streams

Byte streams

- Used for **binary data**: data stored as raw bytes.
- Classes: **`FileInputStream`** (reads bytes) and **`FileOutputStream`** (writes bytes).
- Examples: images, videos, audio files.

Character streams

- Used for **text data**.
- Classes: **`FileReader`** (reads characters) and **`FileWriter`** (writes characters).
- Examples: text files, source code files.

Why have two kinds?

Text is made of characters, and a character must be converted to bytes (using an encoding) when stored. Character streams do that conversion automatically. An image has no characters to convert, so a byte stream passes the bytes through untouched.

> **Note for Review:** Using `FileReader` / `FileWriter` to copy an image can corrupt it, because the stream tries to interpret raw bytes as text. Rule of thumb: **text → character streams, everything else → byte streams**

5. Writing Text Files with FileWriter

The problem

A billing program calculates a message or a result and must save it so it is still there tomorrow. We need to send text from the program into a file.

The general pattern

```
try {
    FileWriter variableName = new FileWriter("fileName");
    variableName.write("text to store");
    variableName.close();
}
catch (IOException e) {
    // handle the problem
}
```

The example

```
import java.io.*;

class WriteFile {
    public static void main(String args[]) {
        try {
            FileWriter fw = new FileWriter("data.txt");
            fw.write("Java File Handling");
            fw.close();
            System.out.println("File Written");
        }
        catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
File Written
```

The file `data.txt` now contains `Java File Handling`.

> The lecture's printed output reads "File Writen". The program as written prints `File Written`.

Syntax breakdown

Part	Meaning
`import java.io.*;`	Makes `FileWriter` and `IOException` available
`class WriteFile`	The class that holds `main`. By convention the class name matches the file name (`WriteFile.java`); this is *mandatory* only for a `public` class
`public static void main(String args[])`	The entry point where execution starts
`try { ... }`	Marks code that might throw an exception
`FileWriter fw`	**Declares** a reference variable `fw` (any valid name works)
`new FileWriter("data.txt")`	**Creates** a `FileWriter` object connected to `data.txt`, and `=` stores its reference in `fw`. If the file does not exist it is created; if it exists, its old contents are replaced
`fw.write("...")`	Sends the text to the file
`fw.close()`	Finishes writing and releases the file
`catch (IOException e)`	Runs only if an `IOException` occurs inside `try`; `e` holds the details

Declaration and creation can also be separated, exactly like any object:

```
FileWriter fw;
fw = new FileWriter("data.txt");
```

The first statement declares a reference variable; the second creates the object (running the constructor) and assigns its reference to `fw`.

Execution trace

Step	Operation	Result
1	`new FileWriter("data.txt")`	`data.txt` is opened for writing; `fw` refers to the writer
2	`fw.write("Java File Handling")`	The text is handed to the writer
3	`fw.close()`	The text is flushed into the file and the file is released
4	`System.out.println("File Written")`	Message appears on the console
5	End of `try`	`catch` block is skipped (no exception occurred)

If step 1 fails (for example, the folder does not exist), Java jumps **directly** to the `catch` block and skips the remaining `try` statements.

Common mistakes

- **Forgetting `close()`.** Every file you open must be closed when you finish. Without it, written text may never reach the file.
- **Forgetting the exception handling.** `FileWriter`'s constructor can throw `IOException`, a *checked* exception. Code that neither catches it nor declares `throws` does not compile.
- **Forgetting `import java.io.*;`.** The compiler cannot find `FileWriter`.

Supplementary: `finally` and appending

`finally` runs whether or not an exception occurred, which makes it the right place for cleanup. It is optional in simple programs, but it guarantees `close()` runs even if `write` fails:

- `fw` is declared **outside** `try` so that `finally` can see it (a variable declared inside `try` is local to that block).
- `fw != null` guards against the case where the constructor itself failed and `fw` was never assigned.
- `close()` can also throw `IOException`, hence the inner `try-catch`.

To **add** to the end of a file instead of replacing it, pass `true` as a second argument: `new FileWriter("data.txt", true)`.^

```
import java.io.*;

class WriteWithFinally {
    public static void main(String args[]) {
        FileWriter fw = null;
        try {
            fw = new FileWriter("data.txt");
            fw.write("Java File Handling");
        }
        catch (IOException e) {
            System.out.println(e);
        }
        finally {
            try {
                if (fw != null) {
                    fw.close();
                }
            }
            catch (IOException e) {
                System.out.println(e);
            }
        }
    }
}
```

---

6. Reading Text Files with FileReader

The problem

The text saved in `data.txt` must come back into the program. `FileReader` fetches it **character by character**.

The example

```
import java.io.*;

class ReadFile {
    public static void main(String args[]) {
        try {
            FileReader fr = new FileReader("data.txt");
            int ch;
            while ((ch = fr.read()) != -1) {
                System.out.print((char) ch);
            }
            fr.close();
        }
        catch (IOException e) {
            System.out.println(e);
        }
    }
}
```

Output (when `data.txt` holds the text written earlier):

```
Java File Handling
```

Syntax breakdown

Part	Meaning
`new FileReader("data.txt")`	Opens `data.txt` for reading and returns a reader object stored in `fr`
`int ch;`	A variable that holds each value returned by `read()`
`fr.read()`	Reads **one** character and returns its numeric (Unicode) value, or `-1` when the file has ended
`(ch = fr.read()) != -1`	**Assigns first, then compares.** The parentheses force the assignment to happen before `!=` is evaluated
`(char) ch`	A **cast** that turns the number back into a character
`System.out.print`	Prints without a newline, so characters appear side by side

Why `int` and not `char`?

`read()` must be able to return **every** valid character *and* a special end-of-file signal. A `char` has no room for a "none left" value, so `read()` returns an `int`: `0` to `65535` for characters, `-1` for the end.

Execution trace

Iteration	`fr.read()` returns	`(char) ch` printed	Loop continues?
1	74	`J`	yes
2	97	`a`	yes
3	118	`v`	yes
4	97	`a`	yes
...	...	...	...
last	-1	(nothing)	**no**, loop ends

FileWriter versus FileReader

	`FileWriter`	`FileReader`
Direction	Program → file	File → program
Main method	`write(...)`	`read()`
Effect on the file	Creates or replaces its contents	Leaves the file unchanged
Stream family	Character stream	Character stream

Common mistakes

- **Declaring `char ch` instead of `int ch`.** Without a cast the code does not compile (an `int` cannot be stored in a `char` without a cast). With a cast, `-1` becomes `65535`, the `!= -1` test is always true, and the loop **never ends**.
- **Forgetting `close()`.**
- **Wrong file name or location.** If `data.txt` does not exist, the constructor throws `FileNotFoundException` (a subclass of `IOException`), so the `catch (IOException e)` block handles it.

---

7. Buffered Streams

The problem

Reading one character at a time means the program keeps asking the disk for tiny pieces of data. Disk access is slow, so this wastes time.

The idea and the analogy

Carrying groceries from the car to the kitchen **one item per trip** is slow. Using a bag to carry many items per trip is faster. A **buffer** is that bag: an area of memory that collects data so the disk is accessed less often.

> The analogy stops at the "bag size": a buffer has a fixed capacity and is filled and emptied automatically, so you never manage individual items yourself.

The formal concept

Buffered streams improve performance by reducing direct disk access. The classes are:

- `BufferedReader` (instead of reading directly with `FileReader`)
- `BufferedWriter` (instead of writing directly with `FileWriter`)

The example: BufferedReader

```
import java.io.*;

class Demo {
    public static void main(String args[]) throws IOException {
        BufferedReader br =
            new BufferedReader(new FileReader("data.txt"));
        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
        br.close();
    }
}
```

Syntax breakdown

Part	Meaning
`new BufferedReader(new FileReader("data.txt"))`	**Wrapping.** The inner `FileReader` opens the file; the outer `BufferedReader` adds a buffer on top of it
`String line;`	Holds one full line of text
`br.readLine()`	Reads an entire line (without the line-ending characters) and returns it as a `String`
`!= null`	`readLine()` returns `null` when there are no more lines
`System.out.println(line)`	Prints the line and moves to the next one

Output (for a file containing `Java File Handling`):

```
Java File Handling
```

`read()` versus `readLine()`

	`read()` (FileReader)	`readLine()` (BufferedReader)
Returns	`int` (one character)	`String` (one line)
End of file signalled by	`-1`	`null`
Loop condition	`!= -1`	`!= null`

`try-catch` versus `throws`

This program uses `throws IOException` on `main` instead of a `try-catch` block.

	`try-catch`	`throws IOException`
Who handles the problem	The method itself	The caller (for `main`, the JVM)
If an `IOException` happens	The `catch` block runs and the program can continue	The program stops and reports the exception
Code size	Longer	Shorter

Supplementary: BufferedWriter

```
import java.io.*;

class BufferedWrite {
    public static void main(String args[]) throws IOException {
        BufferedWriter bw =
            new BufferedWriter(new FileWriter("data.txt"));
        bw.write("Java File Handling");
        bw.newLine();
        bw.write("Buffered streams");
        bw.close();
    }
}
```

`newLine()` writes a line separator appropriate to the operating system. `close()` also empties the buffer into the file.

---

8. Reading and Writing Binary Files

The problem

Not all data is text. Images, videos, and audio files are sequences of **bytes**, not characters. We need streams that move bytes without interpreting them.

The formal concept

Binary files store data in byte format. The classes are:

- **`FileOutputStream`** for writing
- **`FileInputStream`** for reading

Writing binary data

Part	Meaning
`throws IOException`	Declares that this method may throw `IOException` instead of catching it
`new FileOutputStream("data.dat")`	Opens (or creates) the file `data.dat` for writing bytes
`text.getBytes()`	Converts the `String` into an **array of bytes**
`byte b[]`	An array variable that holds those bytes
`fos.write(b)`	Writes the whole byte array to the file
`fos.close()`	Releases the file

```
import java.io.*;

class BinaryWrite {
    public static void main(String args[]) throws IOException {
        FileOutputStream fos = new FileOutputStream("data.dat");
        String text = "Java";
        byte b[] = text.getBytes();
        fos.write(b);
        fos.close();
    }
}
```

The four characters become four bytes:

Character	`J`	`a`	`v`	`a`
Byte value	74	97	118	97
Binary	01001010	01100001	01110110	01100001

Reading binary data

```
import java.io.*;

class BinaryRead {
    public static void main(String args[]) throws IOException {
        FileInputStream fis = new FileInputStream("data.dat");
        int i;
        while ((i = fis.read()) != -1) {
            System.out.print((char) i);
        }
        fis.close();
    }
}
```

`fis.read()` returns one **byte** as an `int` from `0` to `255`, or `-1` at the end of the file. The loop is the same pattern as `FileReader`, but here the values are raw bytes.

> **Note for Review:** The lecture's printed output reads `JAVA`, but the file contains `Java` (written by `BinaryWrite`), so the program prints `Java`. `JAVA` would appear only if the stored text were uppercase.

> **Note for Review:** Casting each byte with `(char)` reproduces the text only for single-byte characters such as ASCII. Characters that need several bytes would be printed incorrectly byte by byte. That is a good reason to use character streams for text.

Output:

```
Java
```

Text files versus binary files

	Text file	Binary file
Classes	`FileReader`, `FileWriter`	`FileInputStream`, `FileOutputStream`
Data unit	Characters	Bytes
Typical contents	`.txt`, source code	Images, video, audio
`read()` returns	Character value, `-1` at end	Byte value (0–255), `-1` at end

---

9. Serialization

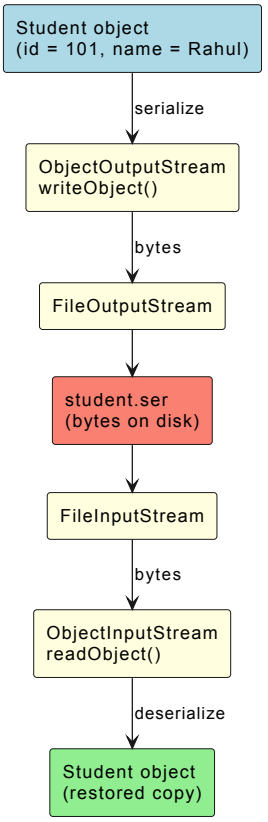
The problem

A `Student` object has an `id` and a `name`. Writing each field to a file separately, and rebuilding the object by hand when reading it back, is tedious and error-prone. We want to save the **whole object** in one step.

The idea and the analogy

Flat-pack furniture is taken apart into pieces, stored or shipped in a box, and later assembled again. **Serialization** takes an object apart into a sequence of bytes. **Deserialization** assembles it again.

> The analogy has one important limit: assembling creates a **new, separate object** with the same data. It is a copy, not the original object.



The formal concept

Serialization is the process of converting an object into a byte stream so that it can be:

- stored in a file,
- sent over a network,
- reconstructed later.

The reverse process is **deserialization**.

The Serializable interface

Java provides `java.io.Serializable`. It is a **marker interface**: it has **no methods**. Implementing it simply tags the class, telling Java "objects of this class may be serialized."

Creating a serializable class

Part	Meaning
`implements Serializable`	The class **implements an interface**. (`extends` is for inheriting from a class; `implements` is for interfaces.) Because the interface has no methods, nothing else must be written
`int id; String name;`	**Fields**: the state stored in each object
`Student(int id, String name)`	A **constructor**: same name as the class, **no return type**, runs when `new Student(...)` is executed. It takes two parameters
`this.id = id;`	Left side `this.id` is the **field**; right side `id` is the **parameter**. `this` removes the ambiguity when they share a name

```
import java.io.*;

class Student implements Serializable {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

`import java.io.*;` already covers `Serializable`, so no extra import is needed.

Writing an object to a file

```
class SerializeDemo {
    public static void main(String args[]) throws Exception {
        Student s = new Student(101, "Rahul");

        FileOutputStream fos = new FileOutputStream("student.ser");
        ObjectOutputStream out = new ObjectOutputStream(fos);

        out.writeObject(s);
        out.close();
        fos.close();
        System.out.println("Object Saved");
    }
}
```

(`import java.io.*;` is needed at the top of the file, as in the `Student` listing above.)

Part	Meaning
`new Student(101, "Rahul")`	Creates the object; the constructor sets `id` and `name`; `s` refers to it
`new FileOutputStream("student.ser")`	A byte stream connected to the file `student.ser` (`.ser` is the usual extension, not a rule)
`new ObjectOutputStream(fos)`	**Wraps** the byte stream with a layer that knows how to turn an object into bytes
`out.writeObject(s)`	Serializes `s` and sends the bytes into the file
`out.close(); fos.close();`	Closes the streams, the outer one first
`throws Exception`	The common form used in this program so the exceptions need not be caught individually

Output:

```
Object Saved
```

Step	Operation	Result
1	`new Student(101, "Rahul")`	Object created with `id = 101`, `name = "Rahul"`
2	`new FileOutputStream("student.ser")`	File opened for writing bytes
3	`new ObjectOutputStream(fos)`	Object-writing layer attached to the file stream
4	`out.writeObject(s)`	Object converted to bytes and written
5	`out.close()`, `fos.close()`	Streams closed
6	`println("Object Saved")`	Message printed

Reading an object from a file

```
import java.io.*;

class DeserializeDemo {
    public static void main(String args[]) throws Exception {
        ObjectInputStream in = new ObjectInputStream(
            new FileInputStream("student.ser"));

        Student s = (Student) in.readObject();

        System.out.println(s.id);
        System.out.println(s.name);

        in.close();
    }
}
```

Part	Meaning
`new ObjectInputStream(new FileInputStream("student.ser"))`	A byte stream from the file, wrapped by a layer that rebuilds objects
`in.readObject()`	Reads the bytes and reconstructs the object
`(Student)`	A **cast**. `readObject()` is declared to return `Object` (the compile-time type), while the object it actually returns is a `Student` (the runtime type). The cast tells the compiler to treat it as a `Student`
`s.id`, `s.name`	The restored values

Output:

```
101
Rahul
```

> **Note for Review:** `throws Exception` is used because reading an object can throw `IOException` *and* `ClassNotFoundException`. `Exception` is the parent of both, so one declaration covers them.

> **Note for Review:** During deserialization the `Student` constructor is **not** run. The object is rebuilt directly from the stored bytes. Also, a class that does not implement `Serializable` cannot be written: `writeObject` throws `NotSerializableException`.

10. The transient Keyword

The problem

A `Student` object might hold a `password`. Saving it in a file is a security risk. We need a way to say "serialize everything **except** this field."

The formal concept

The **`transient`** keyword prevents a variable from being serialized.

```
class Student implements Serializable {
    int id;
    transient String password;
}
```

`password` will **not** be stored.

Syntax

```
transient dataType variableName;
```

`transient` is a **modifier** placed before the type of the field.

What happens on reading back

A transient field is not in the file, so after deserialization it receives its **default value**: `null` for object references such as `String`, `0` for numbers, `false` for `boolean`.

Example variation (the same write and read programs as before, with this class and a constructor `Student(int id, String password)`):

```
class Student implements Serializable {
    int id;
    transient String password;

    Student(int id, String password) {
        this.id = id;
        this.password = password;
    }
}
```

If `new Student(101, "secret123")` is written and then read back, printing the fields gives:

```
101
null
```

Field	Before writing	After reading back
`id`	101	101
`password` (transient)	`"secret123"`	`null`

11. File Handling Summary and Exercises

Summary

- Perform file operations using Java I/O streams.
- Read and write text and binary files.
- Serialize and deserialize Java objects.

Class cheat sheet

Kind	Reads	Writes	Used for
Character stream	`FileReader`	`FileWriter`	Text
Buffered character stream	`BufferedReader`	`BufferedWriter`	Text, with fewer disk accesses
Byte stream	`FileInputStream`	`FileOutputStream`	Binary data
Object stream	`ObjectInputStream`	`ObjectOutputStream`	Whole objects (serialization)

> **Note for Review:** Three habits appear in every program above: import `java.io.*`, handle or declare `IOException`, and **close** what you opened.

Exercises assigned in the lecture

Work through the example programs first, then try these:

- Write a program to copy content from one text file to another.
- Create a program to store and retrieve student records using binary files.
- Develop a serialization program to save and restore employee objects.
- Demonstrate the use of the `finally` block for resource cleanup.
- Implement a file-based student management system using Java I/O.

Part B — Collections and Generics

12. Why We Need Collections

The problem

A college registration program must store student records, but nobody knows in advance how many students will register. Before the Collections Framework, the tool was the array:

```
int arr[] = new int[100];
```

An array like this has three problems:

Problem	What it means
Fixed size	The size is set at creation. If 150 students register, 100 slots are not enough
One type only	This array stores only `int` values. It cannot hold objects of different types
No built-in searching or sorting methods	The array itself offers no ready-made operations for adding, removing, or searching

> **Note for Review:** Arrays *can* hold objects (for example `String[]`) and the `java.util.Arrays` helper class offers some sorting and searching, so "cannot store different object types" refers to this typed array: each array is fixed to one element type. The central problem is that an array has a fixed size and no built-in data-structure behaviour.

The idea and the analogy

An array is a **notebook with exactly 100 pages**: when it is full, you cannot add a page. A collection is a **ring binder**: you add or remove pages freely.

The solution

Collections solve these problems by providing:

- **Dynamic size:** they grow and shrink as needed.
- **Ready-made data structures:** lists, sets, queues, and maps are already written.
- **Efficient operations:** adding, removing, and searching are built in.

---

13. Java Collections Framework

The **Java Collections Framework (JCF)** is a **unified architecture for storing and manipulating groups of objects**. It provides:

- **Interfaces:** the *contracts* (which operations a kind of collection supports),
- **Implementation classes:** the actual working code,
- **Algorithms:** ready-made operations such as sorting and searching.

It lives in the **`java.util`** package:

```
import java.util.*;
```

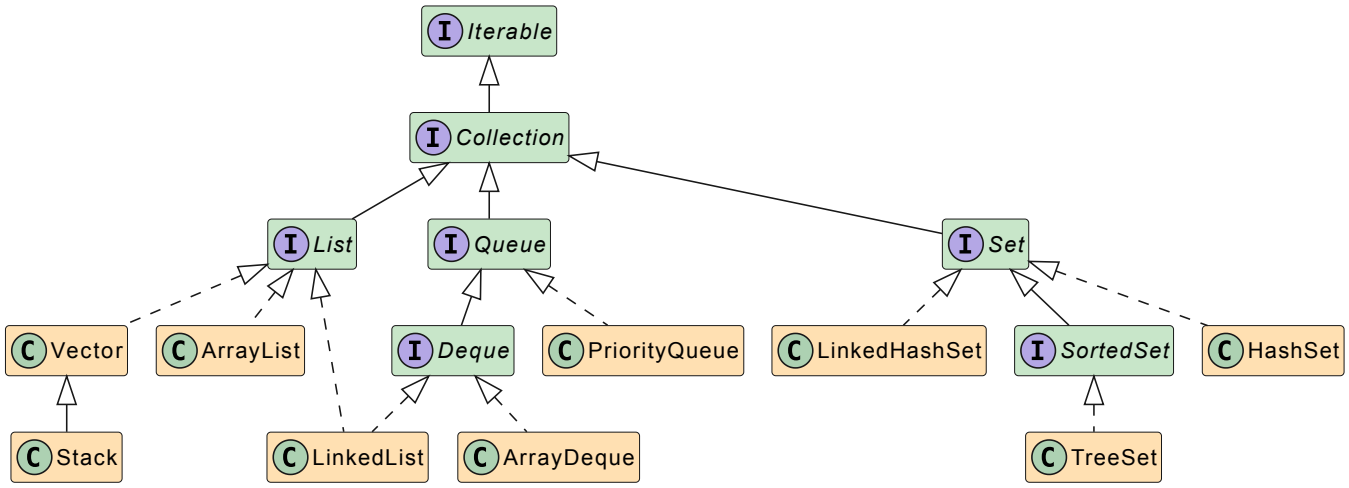
Interface versus class

An **interface** says *what* a collection can do. A **class** says *how* it does it. For example, `List` is an interface; `ArrayList` is a class that implements it.

Reading the architecture diagrams

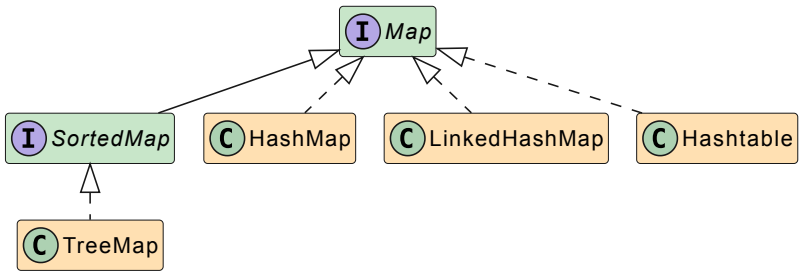
- **Green** boxes are **interfaces**; **orange** boxes are **classes**.
- A **solid** arrow means **extends** (inheritance between interfaces, or between classes).
- A **dotted** arrow means **implements** (a class implementing an interface).

The Collection side



The Map side (a separate hierarchy)

`Map` does **not** extend `Collection`. It stores **key-value pairs**, so it forms its own hierarchy.



> **Note for Review:** The lecture's diagram simplifies two relationships. In real Java, `LinkedHashSet` extends `HashSet` and `LinkedHashMap` extends `HashMap`, so each is attached to its parent class rather than directly to the interface. The diagrams here follow the lecture's simplification.

The architecture at a glance

Interface	Key classes
`Iterable` → `Collection`	The common root of all collections except `Map`
`List`	`ArrayList`, `LinkedList`, `Vector`, `Stack`
`Set`	`HashSet`, `LinkedHashSet`, `SortedSet` → `TreeSet`
`Queue`	`PriorityQueue`, `Deque` → `ArrayDeque`, `LinkedList`
`Map`	`HashMap`, `LinkedHashMap`, `Hashtable`, `SortedMap` → `TreeMap`

`LinkedList` appears under both `List` and `Queue` / `Deque` because it implements both.

The simpler picture used earlier in the lecture shows the same idea in miniature: `Collection` splits into `Set` (with `HashSet`) and `List` (with `ArrayList` and `LinkedList`), while `Map` has `HashMap` and `TreeMap`.

14. The Four Main Interfaces

Everything in the framework rests on four interfaces: **List, Set, Queue, and Map**.

The List interface

A list is like an **attendance register**: entries appear in the order written, the same value may appear twice, and you can ask for "entry number 3".

A `List`:

- **maintains insertion order,**
- **allows duplicate elements,**
- **allows index-based access.**

Examples: `ArrayList`, `LinkedList`, `Vector`, `Stack`.

The Set interface

A set is like the **collection of registered roll numbers**: every roll number must be unique.

A `Set`:

- **does not allow duplicate elements,**
- **does not maintain insertion order** (except `LinkedHashSet`),
- is useful for **unique collections**.

Examples: `HashSet`, `TreeSet`.

> **Note for Review:** `TreeSet` keeps its elements in *sorted* order, which is different from insertion order. `HashSet` makes no ordering promise at all.

The Queue interface

A queue is the **fee-payment counter**: the first student to arrive is the first to be served.

A `Queue` follows **FIFO (First In First Out)**.

Examples: `PriorityQueue`, `LinkedList` (and the double-ended `Deque` family such as `ArrayDeque`).

Used in:

- **Scheduling,**
- **Resource management.**

> **Note for Review:** `PriorityQueue` serves elements by **priority**, not strictly by arrival. FIFO is the general rule of the interface; `PriorityQueue` is the special case where the "front" is the highest-priority element.

The Map interface

A map is like a **register of roll numbers and names**: you look up a student by roll number.

A `Map` stores data as **Key** → **Value** pairs, written `(Key, Value)`:

Key	Value
1	A
2	B
3	C

Properties:

- **Keys must be unique.**
- **Values can be duplicated.**

Examples: `HashMap`, `TreeMap`.

> **Note for Review:** Roll numbers (keys) must be unique, but two students may share the same name (values). `List`, `Set`, and `Queue` hold single values; only `Map` holds key-value pairs.

Comparison

	Order	Duplicates	Accessed by	Examples
List	Insertion order kept	Allowed	Index	`ArrayList`, `LinkedList`, `Vector`, `Stack`
Set	Not kept (`LinkedHashSet` keeps it)	Not allowed	Membership	`HashSet`, `TreeSet`
Queue	FIFO (or priority)	Allowed	Head of the queue	`PriorityQueue`, `LinkedList`
Map	Depends on class	Keys unique, values may repeat	Key	`HashMap`, `TreeMap`

Supplementary: declaration syntax preview

```
import java.util.*;

List<String> names = new ArrayList<String>();
Set<Integer> rollNumbers = new HashSet<Integer>();
Queue<String> feeCounter = new LinkedList<String>();
Map<Integer, String> students = new HashMap<Integer, String>();
```

Pattern: `InterfaceType<ElementType> variable = new ImplementationClass<ElementType>();`

- The **left side** names the interface: what we want the collection to do.
- The **right side** names the class that does it, so changing from `ArrayList` to `LinkedList` changes only one word.
- The part in angle brackets is the **generic type**, explained next.

---

15. Generics

The problem

Consider an `ArrayList` that is meant to hold only cars. Without any restriction it accepts **anything**: a car, a football, a scooter, a fish. Everything that goes in comes out as a plain `Object`, and the programmer must remember what each item really is.

The lecture's picture

	Without generics	With generics
Going in	References to `Car`, `Football`, `Scooter`, `Fish` objects	References to **only** `Car` objects
Coming out	References of type `Object`	References of type `Car`
Declaration	`ArrayList`	`ArrayList<Car>`

The idea and the analogy

A generic collection is a **labelled bin**. A bin labelled "Cars" refuses a football at the door, and everything you pull out is guaranteed to be a car.

Without generics

```
import java.util.*;

class Car { }
class Football { }

class WithoutGenerics {
    public static void main(String args[]) {
        ArrayList garage = new ArrayList();
        garage.add(new Car());
        garage.add(new Football());

        Car first = (Car) garage.get(0);
        Car second = (Car) garage.get(1);
    }
}
```

This **compiles** (with a warning about unchecked use), but the last line **fails at runtime** with a `ClassCastException`, because element `1` is a `Football`. The error is found only when the program runs.

With generics

General syntax:

```
ClassName<Type> variable = new ClassName<Type>();
```

```
ArrayList<Car> cars = new ArrayList<Car>();
cars.add(new Car());
cars.add(new Football());   // INVALID: compile-time error
Car first = cars.get(0);    // no cast needed
```

The invalid line is rejected **before the program runs**, because a `Football` is not a `Car`. And `get(0)` already returns a `Car`, so no cast is required.

(`add(...)` places an element in the list; `get(index)` fetches the element at a position, using the index-based access that `List` provides.)

What generics give you

Benefit	Meaning
Type safety	Wrong types are caught at compile time instead of crashing at runtime
No casting	Elements come out already typed
Code reuse	The same `ArrayList` class works for `Car`, `String`, `Student`, and so on, by changing the type in angle brackets

Generics and primitive types

The type in angle brackets must be a **class**, not a primitive type:

```
ArrayList<int> marks = new ArrayList<int>();         // INVALID
ArrayList<Integer> marks = new ArrayList<Integer>(); // valid
```

Each primitive has a matching **wrapper class** (for example `int` ↔ `Integer`, `double` ↔ `Double`). Wrapper classes are how primitive data is handled inside collections, and they are one of the module's stated objectives.

---

16. Collections and Generics Summary

- The Java Collections Framework is a **unified architecture** of interfaces, implementation classes, and algorithms, found in `java.util`.
- Collections give **dynamic size**, **ready-made data structures**, and **efficient operations**, which fixed-size arrays do not.
- The four main interfaces are **List** (ordered, duplicates, index), **Set** (unique), **Queue** (FIFO), and **Map** (key → value).
- **Generics** make collections type-safe and reusable.

Which collection for which job?

If you need...	Use	Typical class
Ordered items, duplicates allowed, access by position	`List`	`ArrayList`
Only unique items	`Set`	`HashSet` (or `TreeSet` for sorted)
Items processed in arrival order (scheduling, resource management)	`Queue`	`LinkedList`, `PriorityQueue`
Look up a value using a unique key	`Map`	`HashMap` (or `TreeMap` for sorted keys)

Package map

Package	Used for
`java.io`	Input and output
`java.lang`	Core language
`java.util`	Collections Framework

Where to read more

- *A Comprehensive Introduction to Object-Oriented Programming with Java*
- *Java: The Complete Reference*
- NPTEL course material and open course material from universities; solve the exercises at the end of the relevant chapters.

Practice Questions

1. **Find and fix the defects.** The following program is meant to copy `source.txt` into `backup.txt`.

```
import java.io.*;

class CopyFile {
    public static void main(String args[]) {
        FileReader fr = new FileReader("source.txt");
        FileWriter fw = new FileWriter("backup.txt");
        char ch;
        while ((ch = (char) fr.read()) != -1) {
            fw.write(ch);
        }
    }
}
```

   Identify every defect, classify each as a compile-time error, a runtime problem, or a logical problem, and explain what Java would do. Then write a corrected version that still copies the file character by character.

- **Trace the object state.** Study this class and program.

```
class Employee implements Serializable {
    int empId;
    String name;
    transient double bonus;
    transient String pin;

    Employee(int empId, String name, double bonus, String pin) {
        this.empId = empId;
        this.name = name;
        this.bonus = bonus;
        this.pin = pin;
    }
}
```

  An `Employee` created as `new Employee(7, "Meera", 5000.0, "4321")` is written to `emp.ser` with `ObjectOutputStream` and then read back with `ObjectInputStream`. Predict the values of all four fields after reading, and explain why. Then explain what happens, and at which stage, if `implements Serializable` is removed from the class.

- **Bytes in, bytes out.** A program writes the text `"Hi!"` to `msg.dat` using `FileOutputStream` and `getBytes()`. A second program reads the file with `FileInputStream` and, for each byte read, prints the `int` value followed by the `(char)` cast on one line. Predict the exact output and the size of the file in bytes. Then explain why printing `(char)` for each byte could fail to reproduce the original text if the text contained a character that needs more than one byte, and which stream family you would use instead.

- **Choose the right container.** A college system needs: (a) enrolment numbers that can never be repeated; (b) students waiting at the fee counter, served in the order they arrive; (c) a way to find a student's name quickly from a roll number; (d) a list of exam marks where the same mark may occur many times and any mark can be fetched by its position. For each requirement, name the interface and one suitable implementation class, justify your choice from the properties of the interface, and write the generic declaration of the variable. Which requirement would make `ArrayList` a poor choice, and why?

- **Raw list versus generic list.** Examine this code.

```
ArrayList marks = new ArrayList();
marks.add(72.5);
marks.add(88.0);
marks.add("91");
double total = (Double) marks.get(0)
             + (Double) marks.get(1)
             + (Double) marks.get(2);
```

  State whether the problem is detected at compile time or at runtime, which line causes it, and why. Rewrite the declaration using generics and explain exactly what changes about when and where the mistake is reported. Finally, explain why `ArrayList<double>` is not a valid alternative and what must be used instead.